

**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних наук та кібернетики
Кафедра математичної інформатики

Курсова робота
за спеціальністю 122 Комп'ютерні науки
на тему:

**МОНОКУЛЯРНА ВІЗУАЛЬНА ОДОМЕТРІЯ
В РЕАЛЬНОМУ ЧАСІ ДЛЯ БПЛА**

виконала студентка 3-го курсу

ОПП «Інформатика»

Уточкіна Яна Максимівна

(підпис)

Науковий керівник:

завідувач кафедри теоретичної кібернетики

доктор фізико-математичних наук

Терещенко Василь Миколайович

(підпис)

Засвідчую, що в цій роботі
немає запозичень з праць інших авторів
без відповідних посилань.

Студентка

(підпис)

РЕФЕРАТ

Обсяг роботи 28 сторінок, 2 ілюстрацій, 2 таблиць, 14 джерел посилань.

ВІЗУАЛЬНА ОДОМЕТРІЯ, КАЛІБРУВАННЯ КАМЕРИ, МОНОКУЛЯРНА КАМЕРА, ОПТИЧНИЙ ПОТІК, ВЕБ-ЗАСТОСУНОК, КОМП'ЮТЕРНИЙ ЗІР.

Об'єктом роботи є процес оцінювання траєкторії руху монокулярної камери за послідовністю зображень у режимі реального часу або з відеофайлу. Предметом роботи є методи детекції та відстеження ознак, відновлення відносної пози камери та програмна реалізація веб-системи візуальної одометрії.

Метою роботи є розробка та дослідження програмного комплексу монокулярної візуальної одометрії з веб-інтерфейсом для візуалізації тривимірної траєкторії.

Методи розроблення: аналіз літератури з візуальної одометрії, проектування клієнт–серверної архітектури, реалізація алгоритмів комп'ютерного зору (OpenCV), компонентна веб-розробка (Angular), експериментальна перевірка на наборах даних EuRoC MAV. Інструменти: Python, FastAPI, OpenCV, Angular, Plotly.js, Docker.

Результати роботи: спроектовано та реалізовано веб-систему з транспортом WebSocket; розроблено модуль VO на базі Shi–Tomasi, Lucas–Kanade та суттєвої матриці з RANSAC; реалізовано калібрування камери за шаховою дошкою; отримано профіль камери з похибкою репроекції близько 1,21 px на даних EuRoC; реалізовано експорт траєкторії у формат TUM та кількісну оцінку глобального масштабу за ground truth EuRoC (Sim(3)-вирівнювання, АТЕ). Вихідний код і документація розміщені у відкритому репозиторії GitHub [14].

Система може застосовуватися для навчальної демонстрації принципів візуальної одометрії, прототипування алгоритмів локалізації БПЛА та попередньої оцінки траєкторії за відео з бортової камери.

ЗМІСТ

Реферат	1
Скорочення та умовні позначки	4
Вступ	5
1 Теоретичні основи монокулярної візуальної одометрії	7
1.1 Модель камери з центральною перспективою	7
1.2 Калібрування камери	7
1.3 Постановка задачі та класифікація підходів	9
1.4 Детекція та відстеження ознак	10
1.5 Епіпольярна геометрія та суттєва матриця	11
1.6 Неоднозначність масштабу в монокулярній VO	11
1.7 Зв'язок візуальної одометрії та SLAM	11
1.8 Огляд систем PTAM та ORB-SLAM	12
1.8.1 Parallel Tracking and Mapping (PTAM)	12
1.8.2 ORB-SLAM	13
1.8.3 Порівняння з розробленою системою	13
2 Проєктування системи	15
2.1 Загальна архітектура	15
2.2 Режими роботи	15
2.3 Протокол обміну даними	16
2.4 Зберігання конфігурації	16
2.5 Стек технологій	16
3 Реалізація програмного комплексу	17
3.1 Модуль візуальної одометрії	17
3.1.1 Попередня обробка	17
3.1.2 Конвеєр обробки кадру	17
3.1.3 Стани відстеження	18

3.2	Модуль калібрування	19
3.3	Експорт траєкторії та оцінювання масштабу	19
3.4	Клієнтська частина та розгортання	19
4	Експериментальні дослідження	22
4.1	Набір даних EuRoC MAV	22
4.2	Результати калібрування	22
4.3	Оцінювання VO	23
4.4	Обмеження експериментів	25
	Висновки	26
	Перелік джерел посилання	27

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАКИ

API — програмний інтерфейс застосунку (Application Programming Interface);
ATE — абсолютна похибка траєкторії (Absolute Trajectory Error);
БПЛА — безпілотний літальний апарат;
GNSS — глобальна навігаційна супутникова система;
IMU — інерціальний вимірювальний блок;
LK — метод Lucas–Kanade;
RANSAC — RANdom SAmple Consensus, оцінювання з випадковою підвибіркою;
RPE — відносна похибка пози (Relative Pose Error);
SLAM — одночасна локалізація та побудова карти (Simultaneous Localization and Mapping);
VO — візуальна одометрія (Visual Odometry);
YAML — мова серіалізації даних YAML Ain't Markup Language;
EMA — експоненційне ковзне середнє (Exponential Moving Average);
REST — стиль обміну даними HTTP (Representational State Transfer);
Sim(3) — перетворення подібності (масштаб, поворот, зсув) у тривимірному просторі;
TUM — формат файлу траєкторії (Technical University of Munich);
 E — суттєва матриця;
 K — матриця внутрішніх параметрів камери.

ВСТУП

Оцінка сучасного стану об'єкта розроблення.

Визначення положення та орієнтації мобільного робота або безпілотного літального апарата є фундаментальною задачею автономної навігації. У середовищах, де сигнал GNSS недоступний або ненадійний, основним джерелом інформації про рух часто стає бортова відеокамера. Візуальна одометрія дозволяє оцінювати відносний рух камери між послідовними кадрами шляхом інтегрування зміщень та обертань у траєкторію [5]. Монокулярна VO використовує один відеопотік, що спрощує апаратну частину, але створює труднощі через неоднозначність масштабу та накопичення похибки. Сучасні SLAM-системи, зокрема ORB-SLAM [2] та PTAM [3], демонструють високу точність, однак їх повна реалізація є складною для навчальних та прототипних задач.

Актуальність роботи та підстави для її виконання.

Розробка веб-орієнтованої системи, яка поєднує алгоритми комп'ютерного зору на сервері та інтерактивну візуалізацію траєкторії в браузері, актуальна для навчального процесу, швидкого прототипування та попередньої оцінки руху БПЛА за відео. Наявність відкритих наборів даних, зокрема EuRoC MAV [1], дозволяє верифікувати калібрування камери та якість VO.

Мета й завдання роботи.

Метою курсової роботи є розробка та дослідження програмного комплексу монокулярної візуальної одометрії з веб-інтерфейсом, що працює в режимі реального часу або з попередньо записаного відео. Для досягнення мети поставлено такі завдання:

- проаналізувати теоретичні основи монокулярної VO та калібрування камери;
- спроектувати клієнт–серверну архітектуру системи;
- реалізувати модуль оцінювання руху на основі оптичного потоку та епіпольярної геометрії;
- реалізувати калібрування камери за зображеннями шахової дошки;

- забезпечити візуалізацію тривимірної траєкторії та керування профілями камери;
- провести експериментальну перевірку на наборах даних EuRoC MAV, зокрема кількісну оцінку траєкторії за ground truth.

Об'єкт, методи й засоби розроблення.

Об'єктом дослідження є процес оцінювання траєкторії руху монокулярної камери за послідовністю зображень. Предметом — методи детекції ознак, відстеження, відновлення пози та програмна реалізація веб-системи VO. У роботі застосовано методи аналітичної геометрії, комп'ютерного зору (OpenCV), проєкування клієнт–серверних систем, компонентний підхід у веб-розробці (Angular), а також експериментальну верифікацію на еталонних наборах даних.

Можливі сфери застосування.

Створена система може використовуватися для демонстрації принципів VO в навчальному процесі, швидкого прототипування алгоритмів локалізації, попередньої оцінки траєкторії БПЛА та збереження параметрів калібрування камери у профілях для подальших сеансів VO.

Структура роботи.

Робота складається з вступу, чотирьох розділів, висновків та переліку джерел посилання. У розділі 1 викладено теоретичні основи; у розділі 2 — проєкування системи; у розділі 3 — реалізацію; у розділі 4 — експериментальні дослідження.

1 ТЕОРЕТИЧНІ ОСНОВИ МОНОКУЛЯРНОЇ ВІЗУАЛЬНОЇ ОДОМЕТРІЇ

1.1 Модель камери з центральною перспективою

Камеру часто апроксимують центральною перспективною моделлю («камера-обскура», pin-hole). Точка світового простору $\mathbf{P} = (X, Y, Z)^T$ проєктується на зображення через матрицю внутрішніх параметрів K :

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \quad K = \begin{bmatrix} f_u & 0 & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}, \quad (1.1)$$

де f_u, f_v — фокусні відстані в пікселях; c_u, c_v — координати головного пункту; λ — масштабний множник глибини.

На рис. 1.1 схематично показано проєкцію точки світового простору на площину зображення та систему координат OpenCV.

Для точної геометрії враховують дисторсію об'єктива, яку компенсують функцією усунення дисторсії `undistort` у OpenCV [4].

1.2 Калібрування камери

Калібрування — визначення K та коефіцієнтів дисторсії за зображеннями відомого просторового шаблону. Найпоширеніший підхід — шахова дошка (метод Чжана, функція OpenCV `calibrateCamera`):

1. для кожного кадра знаходять внутрішні кути сітки;
2. уточнюють їх субпіксельно;
3. мінімізують похибку репроекції — RMS-відстань між спостереженими та проєктованими кутами.

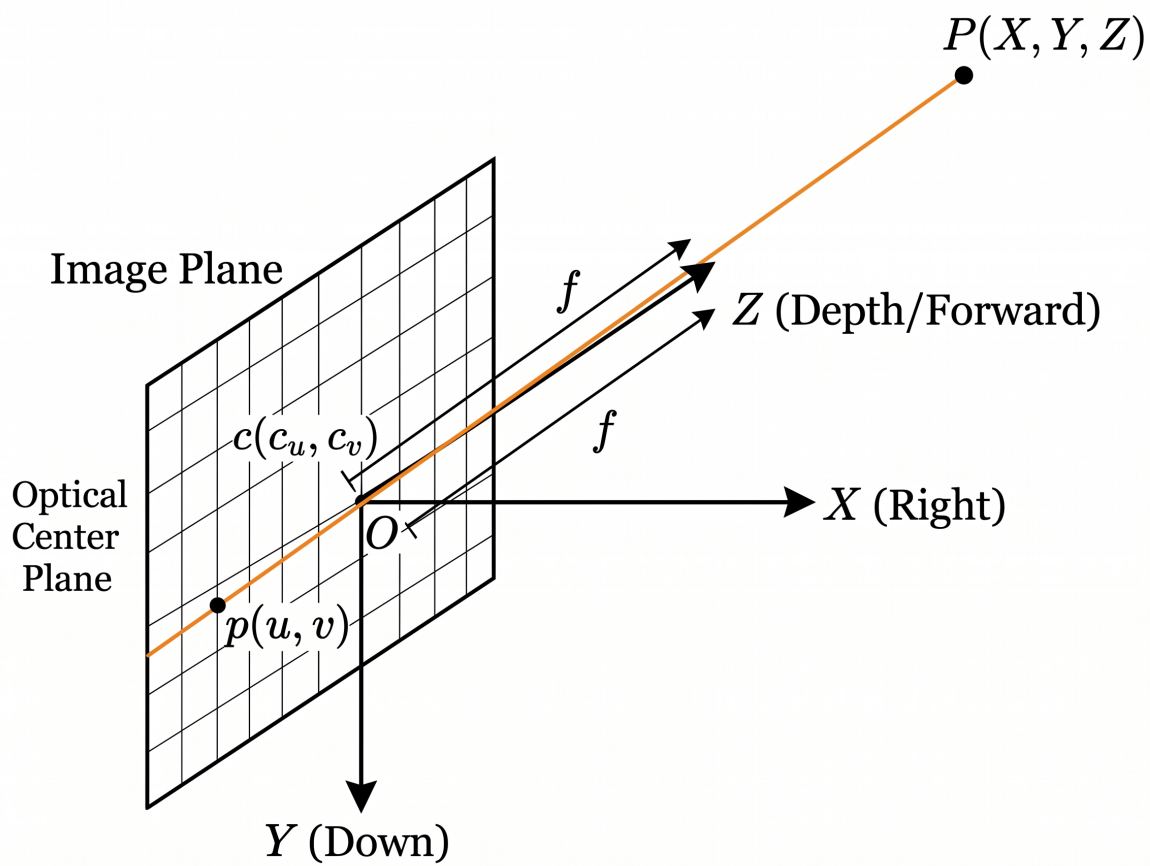


Рис. 1.1 – Модель камери з центральною перспективою та система координат OpenCV

Якість калібрування оцінюють за похибкою репроекції: значення менше 0,5–1,0 px вважають прийнятними для VO.

1.3 Постановка задачі та класифікація підходів

Візуальна одометрія (VO) — оцінка власного руху камери (egomotion) за послідовністю зображень шляхом інкрементального відновлення траєкторії [5], [12]. На відміну від офлайн structure from motion (SFM), VO працює послідовно, у реальному часі. Для ефективної роботи потрібні достатнє освітлення, статична сцена з текстурою та перекриття між сусідніми кадрами; порівняно з колісною одометрією VO дає точнішу оцінку положення (відносна похибка порядку 0,1–2 %) і не залежить від пробуксовування [5].

Формально, для жорсткого перетворення між сусідніми позами камери:

$$T_{k,k-1} = \begin{bmatrix} R_{k,k-1} & \mathbf{t}_{k,k-1} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad C_k = C_{k-1} T_{k,k-1}, \quad (1.2)$$

де $R_{k,k-1}$ — матриця обертання, $\mathbf{t}_{k,k-1}$ — вектор переносу, C_k — поза камери в момент k . Задача VO — обчислити $T_{k,k-1}$ за парою зображень (I_{k-1}, I_k) і накопичити шлях $C_{0:k}$.

За оглядом He та ін. [10], методи монокулярної VO умовно поділяють на три класи: *feature-based* (на основі ознак), *direct* (прямі) та *semi-direct* (напівпрямі).

Методи на основі ознак будують відповідності між ключовими точками та мінімізують похибку репроекції (bundle adjustment, BA). До цього класу належать PTAM [3], ORB-SLAM [2]. Переваги — зрілі алгоритми, зручність для розпізнавання місць і замикання петлі; недолік — витрати на витяг ознак і дескрипторів.

Прямі методи мінімізують фотометричну похибку між пікселями (LSD-SLAM, DSO, DTAM): дають напівщільні карти, але вимагають достатнього освітлення та текстури.

Напівпрямі методи (SVO [11]) поєднують пряме відстеження пікселів з BA на ключових кадрах.

Щодо роботи з ознаками розрізняють *відстеження* (локальний пошук між кадрами, наприклад Lucas–Kanade) і *зіставлення дескрипторів* (краще при вели-

ких змінах ракурсу) [12]. Далі розглянуто алгоритми, реалізовані у розробленій системі.

1.4 Детекція та відстеження ознак

Розроблена система належить до feature-based VO з відстеженням ознак (п. 1.3). Застосовано:

- метод Shi–Tomasi (`goodFeaturesToTrack`) для вибору сильних кутів;
- метод Lucas–Kanade — пірамідальний оптичний потік (`calcOpticalFlowPyrLK`);
- перевірку узгодженості прямого та зворотного потоку — відхилення треку, якщо зворотний потік не повертає точку в початкове положення.

Приклад відстеження ознак між двома послідовними кадрами наведено на рис. 1.2.

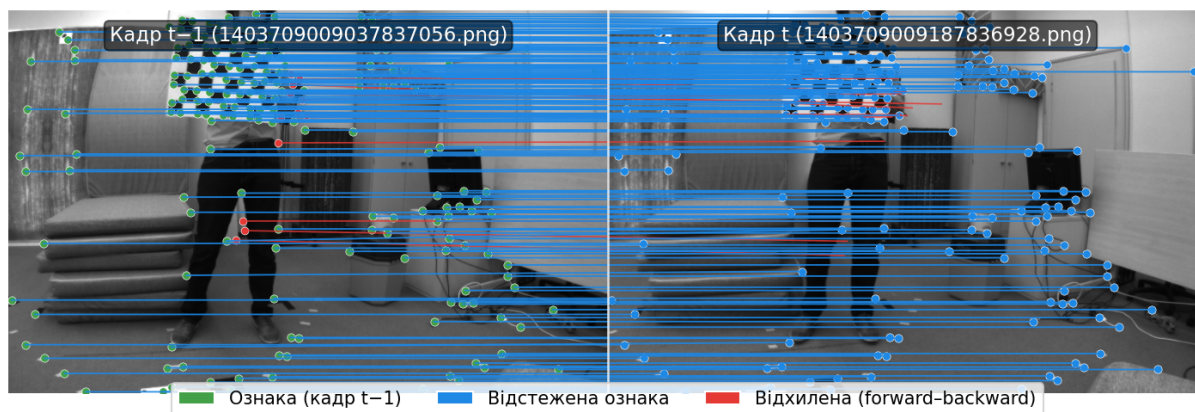


Рис. 1.2 – Відстеження ознак методом Lucas–Kanade між двома кадрами

1.5 Епіполярна геометрія та суттєва матриця

Для двох кадрів з відомою калібровкою K відносний рух описується суттєвою матрицею E [6]:

$$E = [\mathbf{t}]_{\times} R, \quad (1.3)$$

де R — відносне обертання, $\mathbf{t}$ — напрямок переносу (одиниця довжини), $[\mathbf{t}]_{\times}$ — кососиметрична матриця.

Матрицю E оцінюють методом RANSAC (`findEssentialMat`), після чого `recoverPose` відновлює R та $\mathbf{t}$ з перевіркою видимості точок (*cheirality*).

1.6 Неоднозначність масштабу в монокулярній VO

На відміну від стерео- або RGB-D-систем (камера з глибиною), одна камера не спостерігає абсолютну глибину без додаткових припущень. Вектор переносу $\mathbf{t}$ відновлюється лише з точністю до множника [5]. У даній роботі застосовано комбіновану оцінку масштабу кроку (*triangulation*, оптичний потік, адаптивна глибина сцени) та періодичне вирівнювання від ключових кадрів. За наявності еталонної траєкторії (*ground truth EuRoC*) метричний масштаб відновлюється *a posteriori* через *Sim(3)*-вирівнювання (розділ 4).

1.7 Зв'язок візуальної одометрії та SLAM

Візуальна одометрія та *visual SLAM* (*vSLAM*) тісно пов'язані, але вирішують різні аспекти локалізації [10]. VO (формула (1.2)) фокусується на *локальній* узгодженості траєкторії. SLAM додатково будує карту середовища і прагне до *глобальної* узгодженості, зокрема через замикання петлі (*loop closure*).

Для підвищення точності VO застосовують локальну оптимізацію на вікні з m останніх кадрів — *sliding-window bundle adjustment* [10], [12]. SLAM розширює VO модулем карти та глобальної оптимізації. PTAM [3] і ORB-SLAM [2] показали,

що SLAM у реальному часі можливий, якщо ВА виконувати лише на *ключових кадрах*, а не на кожному кадрі — це ефективніше за фільтрові підходи (EKF-SLAM) [2].

Пакетне уточнення (ВА) одночасно уточнює пози камер і координати точок карти. Для точних результатів потрібні [2]: мережа відповідностей між keyframes, значний паралакс, початкові оцінки поз і локальна оптимізація. **Замикання петлі** оцінює накопичений дрейф і вирівнює траєкторію через pose graph; у монокулярному SLAM обмеження мають 7 DoF — перетворення $\text{Sim}(3)$ [2].

1.8 Огляд систем PTAM та ORB-SLAM

1.8.1 Parallel Tracking and Mapping (PTAM)

Система PTAM (Klein, Murray, 2007) [3] — перша широко відома keyframe-based monocular SLAM, що працювала в реальному часі для доповненої реальності. Головна ідея — *паралельне* відстеження (tracking) і побудова карти (mapping) у двох потоках:

- **Tracking** — локалізація камери на *кожному* кадрі: відповідності FAST-кутів з патч-кореляцією, оцінка пози відносно локальної карти;
- **Mapping** — у фоновому потоці: вибір ключових кадрів, тріангуляція нових точок, ВА для уточнення структури та поз у локальній області.

PTAM запровадила практичні прийоми, актуальні й сьогодні: відбір ключових кадрів, тріангуляція, relocalization після втрати відстеження. Обмеження оригінальної версії [2], [3]: робота переважно в невеликих сценах; відсутність замикання великих петель; слабка relocalization (кореляція мініатюр ключових кадрів, низька інваріантність до зміни ракурсу); ручна ініціалізація карти. Пізніші модифікації додали edge-ознаки та покращену relocalization, але архітектура залишилася еталоном для ORB-SLAM.

1.8.2 ORB-SLAM

ORB-SLAM (Mur-Artal та ін., 2015) [2] розвиває ідеї PTAM, додаючи масштабованість, автоматичну ініціалізацію та замикання петлі. Система використовує *одні й ті самі* ORB-ознаки для відстеження, побудови карти, relocalization і loop closure — це спрощує архітектуру та підвищує надійність.

ORB-SLAM працює у **трьох паралельних потоках** [2]:

1. **Tracking** — локалізація кожного кадру: спочатку motion-only BA відносно попереднього кадру; при втраті — глобальна relocalization через розпізнавання місця (DBoW2); далі уточнення пози за локальною картою з *графа співвидимості* (covisibility graph) ключових кадрів; рішення про вставку нового keyframe.
2. **Local mapping** — обробка нових keyframes: локальне BA, пошук нових відповідей, тріангуляція точок, суворя політика відбору («survival of the fittest») для точок і keyframes.
3. **Loop closing** — пошук петель; при виявленні — оцінка Sim(3)-дрейфу, злиття дублікатів точок, глобальна оптимізація *Essential Graph* (розріджений підграф covisibility graph).

Ініціалізація карти автоматична: вибір між гомографією (плоска сцена) та фундаментальною матрицею (загальна 3D-сцена) за евристикою model selection [2]. ORB-дескриптори інваріантні до обертання та масштабу в обмеженому діапазоні, що дозволяє широкобазове зіставлення без GPU.

Наступник ORB-SLAM3 [13] розширює бібліотеку до visual-inertial SLAM, multi-тар і кількох типів камер, зберігаючи feature-based архітектуру з трьома потоками.

1.8.3 Порівняння з розробленою системою

На відміну від PTAM та ORB-SLAM розроблена веб-система реалізує *полегшений VO-конвеєр* без глобальної карти, замикання петлі та повномасштабного BA:

- відстеження Shi–Tomasi + Lucas–Kanade замість ORB-дескрипторів;
- оцінка руху через E + RANSAC (резерв — affine);
- keyframe snap і комбінована оцінка масштабу замість локального BA;
- транспорт WebSocket для навчальної демонстрації в браузері.

Такий вибір зменшує складність розгортання та дозволяє прозоро показати принципи VO; за точністю система поступається повноцінним SLAM, що підтверджується експериментами на EuRoC (розділ 4).

2 ПРОЄКТУВАННЯ СИСТЕМИ

2.1 Загальна архітектура

Система побудована за архітектурою клієнт–сервер (рис. 2.1). Обчислювально складні операції комп’ютерного зору виконуються на сервері (Python); клієнт на Angular відповідає за захоплення відео, налаштування та візуалізацію.

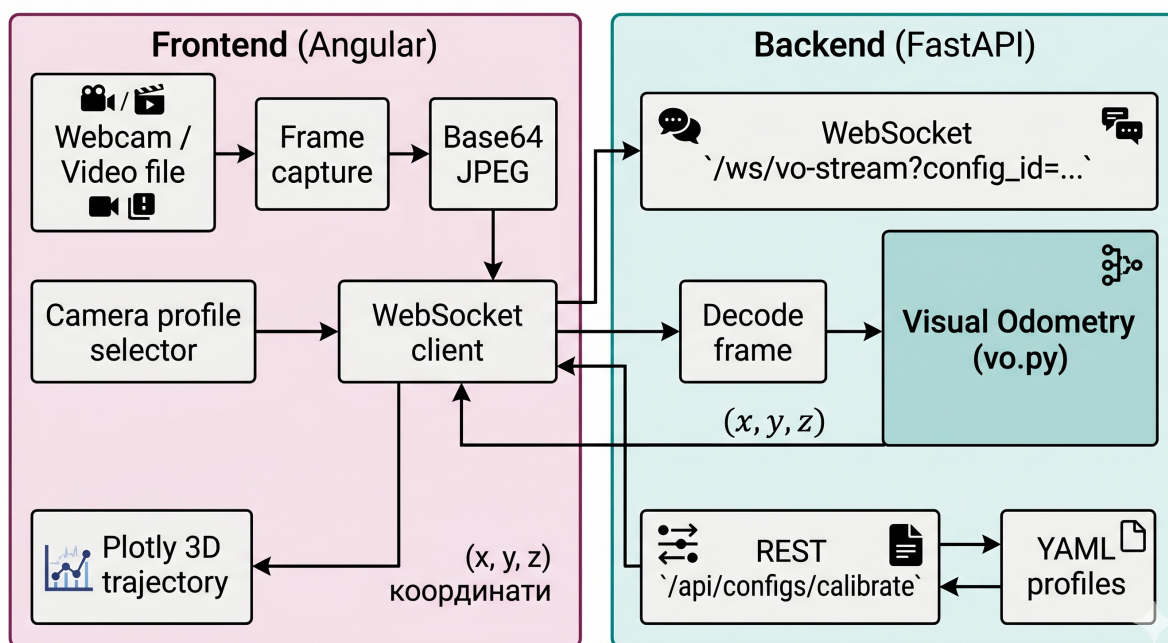


Рис. 2.1 – Логічна архітектура системи

2.2 Режими роботи

Підтримуються два режими: «Потік» — захоплення з вебкамери у реальному часі; «З файлу» — обробка завантаженого відеофайлу (наприклад, конвертованої послідовності EuRoC). Перед запуском VO користувач обирає профіль калібрування камери.

2.3 Протокол обміну даними

З'єднання встановлюється через WebSocket за адресою `ws://localhost:8000/ws/vo-stream?config_id=<profile_id>`. Цикл «запит–відповідь»: клієнт надсилає кадр (JPEG у кодуванні Base64); сервер обробляє кадр у окремому потоці (`asyncio.to_thread`); сервер повертає координати та стан відстеження; клієнт додає точку на тривимірний графік і надсилає наступний кадр.

2.4 Зберігання конфігурації

Профілі камери зберігаються у форматі YAML у каталозі `vo-uav/storage/configs/`. Профіль містить внутрішні параметри f_u , f_v , c_u , c_v , коефіцієнти дисторсії, метадані калібрування та параметри VO.

2.5 Стек технологій

Стек технологій наведено в табл. 2.1.

Табл. 2.1 – Технологічний стек системи

Шар	Технології
Серверна частина	Python 3.11+, FastAPI, OpenCV, NumPy, PyYAML
Клієнтська частина	Angular 21, TypeScript, Angular Material, Plotly.js
Транспорт	WebSocket, REST (калібрування, профілі, експорт траєкторії)
Розгортання	Docker, Docker Compose

3 РЕАЛІЗАЦІЯ ПРОГРАМНОГО КОМПЛЕКСУ

Програмний комплекс реалізовано мовами Python (сервер) та TypeScript (клієнт). Структура проєкту: каталоги `vo-uav/` (сервер FastAPI, модулі VO та калібрування), `vo-frontend/` (Angular-клієнт), `scripts/` (допоміжні утиліти), файл `docker-compose.yml` для розгортання. Повний вихідний код, інструкції зі збирання та запуску опубліковано у репозиторії GitHub [14]: <https://github.com/umunevich/visual-odometry>.

3.1 Модуль візуальної одометрії

Клас `VisualOdometry` (`vo-uav/src/vo.py`) реалізує інкрементальний VO-конвеєр. Система координат OpenCV: X — вправо, Y — вниз, Z — углиб сцени.

3.1.1 Попередня обробка

Виконується масштабування K до фактичної роздільності кадру (якщо вона відрізняється від калібрувальної) та опційне усунення дисторсії за коефіцієнтами з профілю.

3.1.2 Конвеєр обробки кадру

Для кожного кадру I_t :

1. відстеження ознак між I_{t-1} та I_t (LK + прямий–зворотний потік);
2. оцінка відносної пози: основний шлях — $E + \text{RANSAC} + \text{recoverPose}$; ре-

зервний — `estimateAffinePartial2D` для плоских рухів;

3. оцінка масштабу кроку: комбінація відношення глибин (triangulation), оптичного потоку та номінальної глибини сцени; додатковий масштаб з базової лінії `keyframe`→поточний кадр;
4. інтеграція пози; періодичне вирівнювання (snap) від `keyframe` для зменшення довгострокового дрейфу; поповнення ознак;
5. опційне згладжування траєкторії для відображення (ЕМА + відсіювання стрибків).

Блок-схема конвеєра наведена на рис. 3.1.

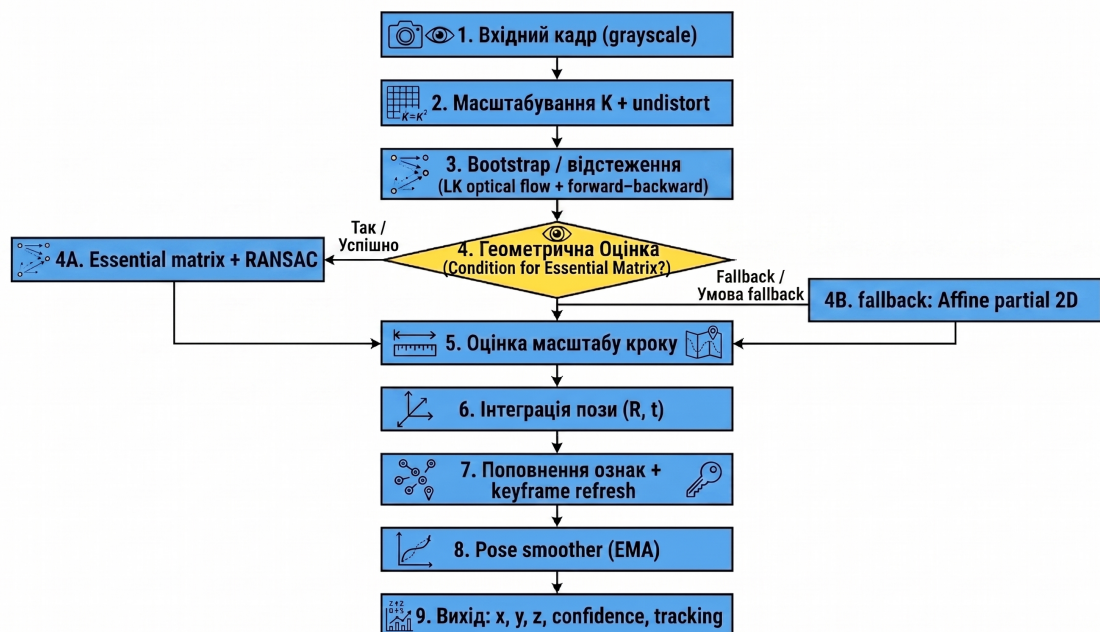


Рис. 3.1 – Блок-схема конвеєра візуальної одометрії

3.1.3 Стани відстеження

Програмний інтерфейс повертає поле `tracking` зі значеннями `initializing`, `ok`, `degraded`, `lost` (ініціалізація, успішно, погіршено, втрачено) для діагностики якості оцінки в інтерфейсі користувача.

3.2 Модуль калібрування

Файл `camera_calibration.py` реалізує побудову тривимірного шаблону кутів шахової дошки, детекцію з `findChessboardCornersSB` та класичним резервним методом, автоматичний підбір розміру сітки (зокрема 6×7 для EuRoC `cam_checkerboard`), виклик `calibrateCamera` та збереження результату. Кінцева точка REST: `POST /api/configs/calibrate`.

3.3 Експорт траєкторії та оцінювання масштабу

Модуль `trajectory_eval.py` забезпечує експорт накопиченої траєкторії у форматі TUM (`timestamp tx ty tz qx qy qz qw`) та кількісне порівняння з еталоном EuRoC. Для кожного кадру VO зберігається індекс кадру; `timestamp` береться з `mav0/cam0/data.csv` відповідної послідовності. `Ground truth` інтерполюється з `state_groundtruth_estimate0/data.csv` на ці мітки часу.

Глобальний масштаб s обчислюється алгоритмом Umeyama (Sim(3)-вирівнювання): еталонна траєкторія $\approx s \cdot R \cdot \mathbf{p}_{VO} + \mathbf{t}$. Після масштабування обчислюється абсолютна похибка траєкторії ATE. REST-кінцеві точки: `GET /api/trajectory/euroc-sequences`, `POST /api/trajectory/export-t`, `POST /api/trajectory/evaluate`. У вкладці «З файлу» доступні кнопки *Export TUM*, *Export scaled TUM* та *Compute global scale*. Для пакетної оцінки використовується скрипт `scripts/run_vo_eval.py`.

3.4 Клієнтська частина та розгортання

Angular-додаток містить вкладки «Потік» / «З файлу», елементи керування профілем камери, робочі простори VO з тривимірним графіком Plotly, вибір послідовності EuRoC для порівняння з еталоном та сервіс WebSocket зі сторожовим

механізмом (watchdog). Інтерфейс показано на рис. 3.2–3.4. Каталог Datasets змонтовано в контейнер сервера (EUROC_DATASETS_PATH=/datasets). Розгортання здійснюється через `docker-compose.yml` (сервер — порт 8000, клієнт — порт 4200).

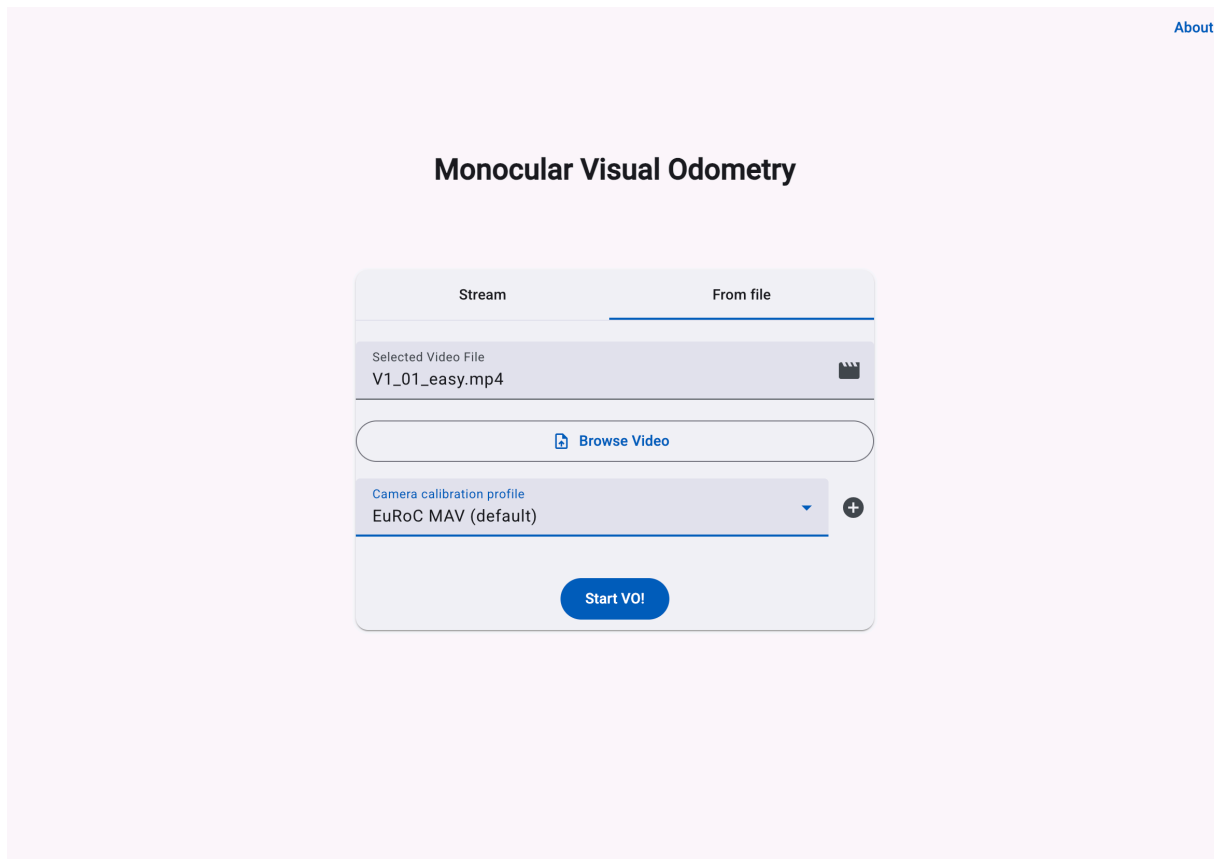


Рис. 3.2 – Головний інтерфейс системи (вкладка «З файлу»)

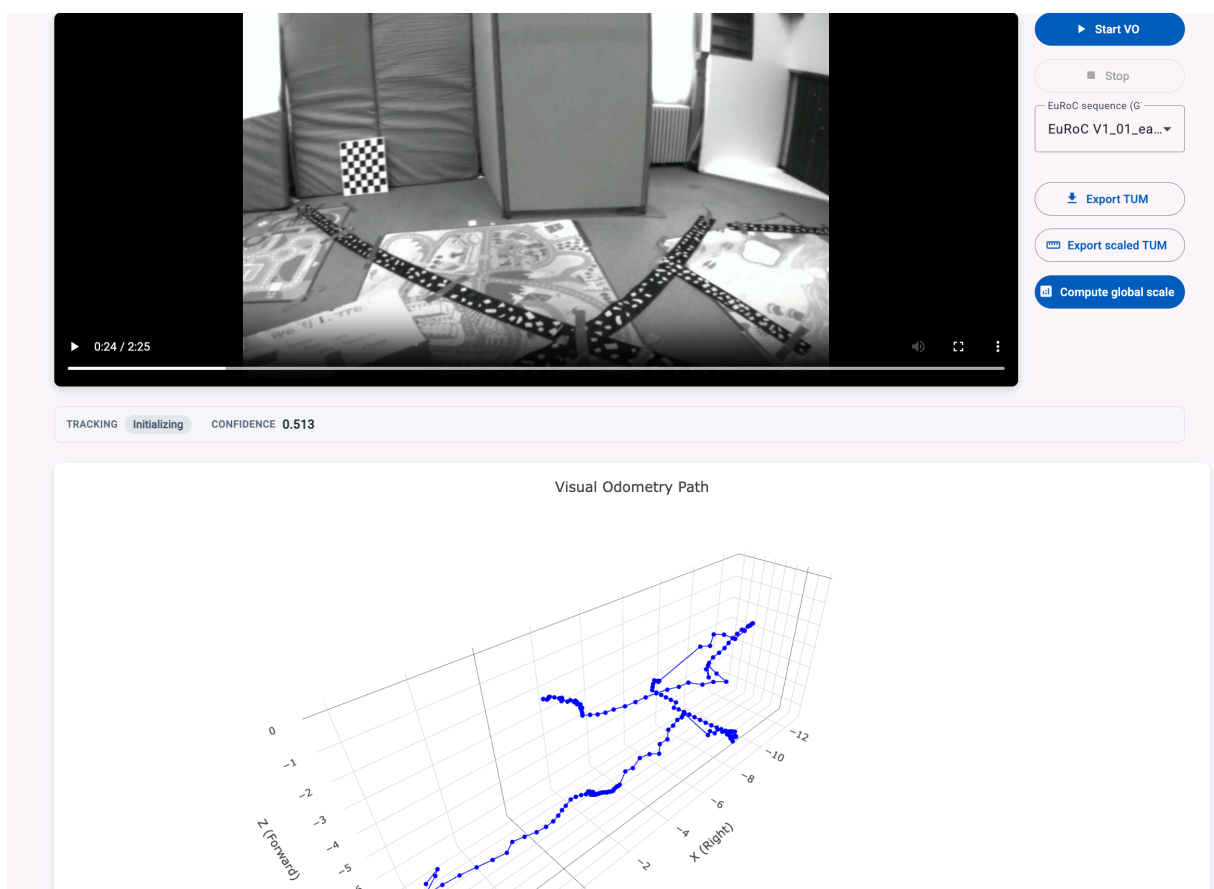


Рис. 3.3 – Робочий простір

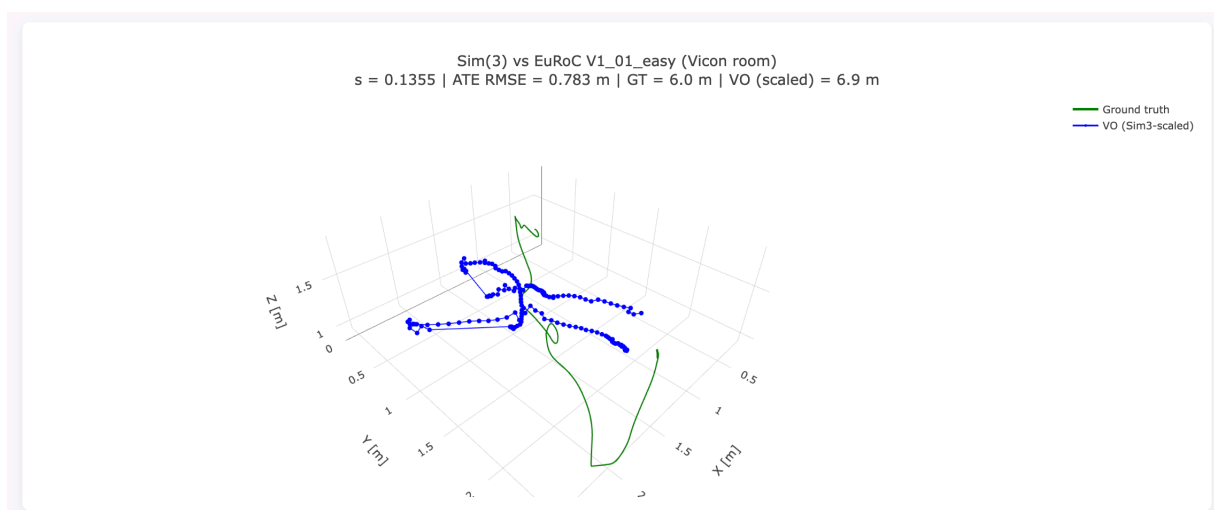


Рис. 3.4 – Обчислення глобального масштабу

4 ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ

4.1 Набір даних EuRoC MAV

Для тестування використано набір EuRoC MAV Dataset (ETH Zurich) [1] — стандартний еталон для візуально-інерційної одометрії та SLAM. Послідовність `cam_checkerboard` містить кадри камери `cam0` (752×480 px, 20 Гц) з видимою шаховою дошкою. Для калібрування потрібна сітка 6×7 внутрішніх кутів. З 2376 кадрів дошку успішно детектовано на 1740.

4.2 Результати калібрування

Параметри профілю `dataset.yaml`, отриманого за 11 зображеннями з EuRoC, наведено в табл. 4.1.

Табл. 4.1 – Параметри калібрування (профіль Dataset)

Параметр	Значення
f_u	397,88 px
f_v	394,66 px
c_u	360,80 px
c_v	271,66 px
Похибка репроекції	1,21 px
Роздільність	752 × 480 px
Внутрішні кути	6 × 7
Розмір клітини	38 mm

Похибка репроекції близько 1,2 px — прийнятний рівень; для підвищення точності VO рекомендується більше різноманітних ракурсів. Приклад детекції кутів шахової дошки наведено на рис. 4.1.



Рис. 4.1 – Детекція внутрішніх кутів шахової дошки 6×7 (EuRoC cam_checkerboard)

4.3 Оцінювання VO

Якісна оцінка включає аналіз форми траєкторії на Plotly (рис. 4.2), полів *confidence* (впевненість) та *tracking* (стан відстеження), поведінку при нерухомій камері та порівняння типового профілю EuRoC і власної калібровки.

Для кількісної оцінки на польових послідовностях EuRoC Vicon Room використано еталонну траєкторію (ground truth) з файлу `state_groundtruth_estimates`. Алгоритм оцінювання:

1. експорт траєкторії VO у формат TUM;
2. інтерполяція ground truth на timestamp камери cam0;
3. Sim(3)-вирівнювання (Umeyama) та обчислення глобального масштабу s ;
4. обчислення ATE (RMSE, median, max) між масштабованою та еталонною траєкторіями.

Експеримент проведено на відео `V1_01_easy.mp4` (2912 кадрів, 752×480 px, 20 Гц), отриманому з кадрів cam0 EuRoC, з профілем `euroc_default`. Резуль-

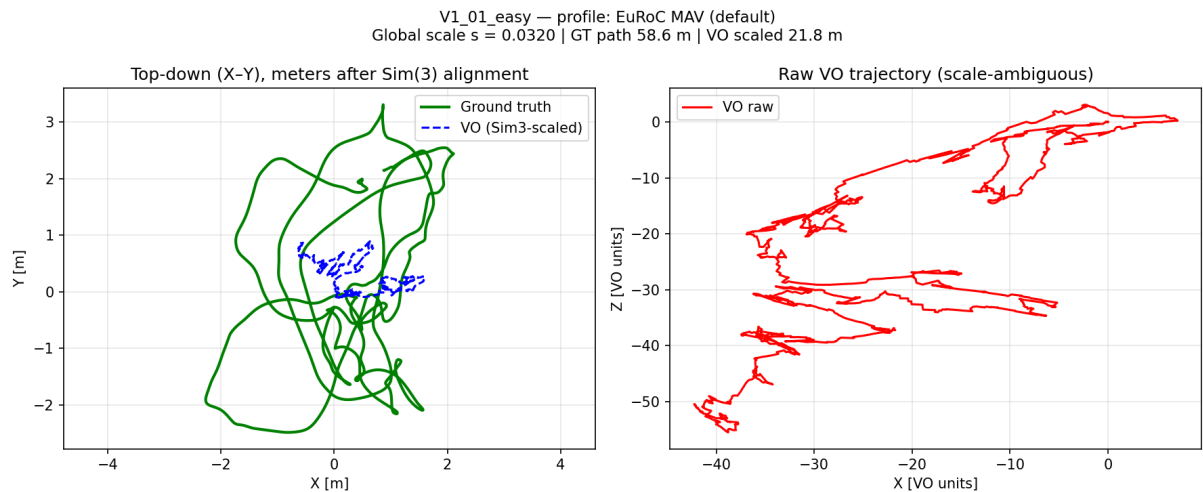


Рис. 4.2 – Тривимірна траєкторія, побудована системою VO (графік Plotly scatter3d)

тати наведено в табл. 4.2.

Табл. 4.2 – Кількісна оцінка VO на послідовності EuRoC v1_01_easy

Метрика	Значення
Кадрів у стані ok	99,9 %
Середня confidence	0,77
Глобальний масштаб s (Sim(3))	0,061
ATE RMSE	1,74 м
ATE median	1,57 м
Довжина шляху еталону	58,6 м
Довжина шляху VO (масштабована)	24,1 м

Після покращення конвеєра (комбінована оцінка масштабу, keyframe baseline, pose snap) довжина Sim(3)-вирівнюваної траєкторії зросла з ≈ 9 м (початкова версія) до 24,1 м при незмінному порядку АТЕ. Високий відсоток кадрів у стані ok свідчить про стабільність відстеження ознак. АТЕ порядку 1,7 м на шляху ≈ 59 м є типовим для монокулярного VO без замикання петлі. Часткове недооцінювання довжини (24 м проти 58,6 м, ≈ 41 % еталону) підтверджує накопичення дрейфу масштабу та форми — обмеження монокулярного VO; повний метричний збіг потребує VIO або замикання петлі.

Додатково траєкторії можна порівнювати з evo [7] (RPE, детальні графіки помилок) за експортованими TUM-файлами.

4.4 Обмеження експериментів

Монокулярна VO без замикання петлі накопичує дрейф; масштаб траєкторії під час польоту залишається умовним — метричний масштаб відновлюється лише *a posteriori* за наявності еталону; сценарії з вебкамерою часто активують резервний affine-метод; затримка мережі WebSocket впливає на частоту кадрів у режимі «Потік».

ВИСНОВКИ

У курсовій роботі спроектовано та реалізовано веб-систему монокулярної візуальної одометрії з клієнт–серверною архітектурою, транспортом WebSocket і тривимірною візуалізацією траєкторії.

Розроблено VO-модуль на базі Shi–Tomasi, Lucas–Kanade, суттєвої матриці з RANSAC, резервного affine-методу та післяобробки згладжування. Реалізовано калібрування камери за шаховою дошкою з автопідбором розміру сітки та збереженням профілів YAML.

На даних EuRoC `cam_checkerboard` отримано профіль з похибкою репроекції близько 1,21 px; підтверджено коректність детекції дошки 6×7 . На послідовності `V1_01_easy` (2912 кадрів) після покращення конвеєра VO: Sim(3)-масштаб $s \approx 0,061$, ATE RMSE $\approx 1,74$ м, довжина масштабованої траєкторії ≈ 24 м ($\approx 41\%$ еталону); стабільність відстеження — 99,9% кадрів у стані ok. Вихідний код опубліковано у репозиторії GitHub [14].

Система придатна для навчальної демонстрації VO та прототипування. Перспективи розвитку: метрики RPE через evo, локальне пакетне уточнення, ORB-дескриптори, поєднання з IMU, замикання петлі.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Burri M., Nikolic J., Gohl P. [та ін.]. The EuRoC micro aerial vehicle datasets. International Journal of Robotics Research. 2016.
2. Mur-Artal R., Montiel J., Tardós J. D. ORB-SLAM: a versatile and accurate monocular SLAM system. IEEE Transactions on Robotics. 2015.
3. Klein G., Murray D. Parallel tracking and mapping for small AR workspaces. ISMAR. 2007.
4. Camera calibration with OpenCV // OpenCV Documentation. URL: https://docs.opencv.org/4.x/d4/d94/tutorial_camera_calibration.html (дата звернення: 29.05.2026).
5. Scaramuzza D., Fraundorfer F. Visual Odometry: Part I — The First 30 Years and Fundamentals. IEEE Robotics & Automation Magazine. 2011.
6. Hartley R., Zisserman A. Multiple View Geometry in Computer Vision. Cambridge University Press, 2004. 655 с.
7. evo: Python package for the evaluation of odometry and SLAM // GitHub. URL: <https://github.com/MichaelGrupp/evo> (дата звернення: 29.05.2026).
8. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата звернення: 29.05.2026).
9. Angular Documentation. URL: <https://angular.dev/> (дата звернення: 29.05.2026).
10. He M., Zhu C., Huang Q. [та ін.]. A review of monocular visual odometry. The Visual Computer. 2019.
11. Forster C., Pizzoli M., Scaramuzza D. SVO: Fast semi-direct monocular visual odometry. IEEE International Conference on Robotics and Automation. 2014.
12. Fraundorfer F., Scaramuzza D. Visual Odometry: Part II — Matching, Robustness, Optimization, and Applications. IEEE Robotics & Automation Magazine. 2012.

13. Mur-Artal R., Tardós J. D. ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM. IEEE Transactions on Robotics. 2021.
14. Уточкіна Я. М. Monocular visual odometry: веб-система візуальної одометрії [Електронний ресурс]. Репозиторій GitHub. URL: <https://github.com/munevich/visual-odometry> (дата звернення: 31.05.2026).